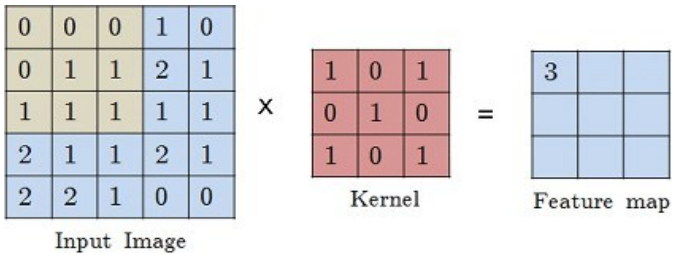
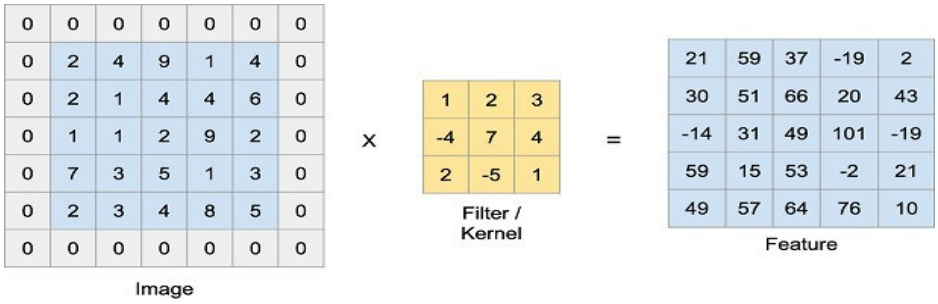
TPN°2 : Filtrage d'images

1. Les noyaux / matrice de convolutions

En traitement d'images, un noyau, une matrice de convolution ou un masque est une petite matrice utilisée pour le floutage, l'amélioration de la netteté de l'image, le gaufrage, la détection de contours, et d'autres. Tout cela est accompli en faisant une convolution entre le noyau et l'image.



Ajout de paddind pour conserver la taille de l'image de sortie



Question 1.Rédiger une fonction convolution qui prend en arguments deux matrices M (associée à une image) et C et retourne la matrice M ⊗ C. Les éléments de la matrice C seront a priori de type float tandis que ceux des matrices M et M ⊗ C seront de type np.uint8. Il conviendra donc, lorsque m∗ij < 0 ou m*ij > 255 de remplacer les valeurs calculées par respectivement 0 et 255. Ne pas oublier non plus que les pixels d'une image en couleur possèdent trois composantes et que chacune de ces composantes doit être traitée.

```
def convolution(m, c):
    n, p, q = m.shape
    mm = np.zeros_like(m)
    for i in range(1, n-1):
        for j in range(1, p-1):
            s = 0
            for u in range(3):
                for v in range(3):
                    s += c[u, v] * m[i-1+u, j-1+v]
            for k in range(q):
                if s[k] < 0:
                    mm[i, j, k] = 0
                elif s[k] > 255:
                    mm[i, j, k] = 255
                else:
                    mm[i, j, k] = s[k]
    return mm
```

Exemples de masques :

- Lissage Pour obtenir un effet de lissage on utilise le masque C1 : chaque pixel est remplacé par la moyenne de lui-même et de ses huit voisins ; pour cette raison on parle de filtre moyenneur.
- Augmentation du contraste Au contraire, pour augmenter le contraste on utilise le masque C2.
- Repoussage Enfin, le masque C3 donne un effet de relief à l'image, appelé repoussage.

$$C_1 = \frac{1}{9}\begin{pmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{pmatrix} \qquad C_2 = \begin{pmatrix}0 & -1 & 0\\ -1 & 5 & -1\\ 0 & -1 & 0\end{pmatrix} \qquad C_3 = \begin{pmatrix}-2 & -1 & 0\\ -1 & 1 & 1\\ 0 & 1 & 2\end{pmatrix}$$

Question 2. Rédiger trois fonctions lissage, contraste et repoussage qui prennent toutes trois en argument une image et retournent une nouvelle image obtenue en appliquant respectivement un effet de lissage, d'augmentation de contraste et de repoussage, et observer le résultat sur l'image test.

```
def lissage(im):
    c = .......
    return convolution(im, c)
```

```
def repoussage(im):
    c = .........
    return convolution(im, c)
```

```
plt.imshow(lissage(image)) #X et Y sont les listes (ou tableaux numpy) d'abscisses e
plt.show() #pour afficher
```

```
plt.imshow(contraste(image)) #X et Y sont les listes (ou tableaux numpy) d'abscisses
plt.show() #pour afficher
```

```
plt.imshow(repoussage(image)) #X et Y sont les listes (ou tableaux numpy) d'abscisse
plt.show() #pour afficher
```

```
def contraste(im):
    c = ........
    return convolution(im, c)
```

2. Ajout de bruit à une image

Générer du bruit "poivre et sel" (salt and pepper) sur une image.

1. Exécutez le programme plusieurs fois pour voir si le bruit change?
2. Afficher l'histogramme de l'image bruitée?

Complétez le programme suivant pour ajouter du bruit poivre et sel à une image :

```
import cv2
import numpy as np import random
import matplotlib.pyplot as plt

# Charger l'image
img = cv2.imread('aero1.jpg', 0)
# Créer une copie pour le bruit
noisy_img = img.copy()
# Récupérer les dimensions
row, col = noisy_img.shape

# BRUIT BLANC (Salt)
# À compléter : ajouter entre 300 et 10000 pixels blancs (valeur 255)
number_of_white_pixels = random.randint(300, 10000)
for i in range(number_of_white_pixels):
# Choisir des coordonnées aléatoires
y_coord = random.randint(0, row - 1)
x_coord = random.randint(0, col - 1)
 # Mettre le pixel à 255 (blanc)
noisy_img[y_coord][x_coord] = 255

# BRUIT NOIR (Pepper)
# À compléter : ajouter entre 300 et 10000 pixels noirs (valeur 0) # Même logique que pour
    le bruit blanc
# Afficher l'image originale et l'image bruitée
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].imshow(img, cmap='gray')
ax[0].set_title('Image originale')
ax[0].axis('off')

ax[1].imshow(noisy_img, cmap='gray')
ax[1].set_title('Image avec bruit Salt & Pepper')
ax[1].axis('off')
plt.show()

# Sauvegarder l'image bruitée
cv2.imwrite('noisy_image.png',noisy_img)
```

3. Filtre moyenneur

- Appliquer un filtre moyenneur pour réduire le bruit.
- Testez différentes tailles de filtres moyenneurs sur l'image bruitée.

- Méthode 1 : Utilisation de cv2.filter2D()

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Charger l'image bruitée
img = cv2.imread('noisy_image.png', 0)
# Définir un filtre moyenneur 3x3
kernel_3x3 = np.ones((3, 3), np.float32) / 9

# Définir un filtre moyenneur 5x5
kernel_5x5 = # À compléter
# Définir un filtre moyenneur 7x7
kernel_7x7 = # À compléter
# Appliquer les filtres
filtered_3x3 = cv2.filter2D(img, -1, kernel_3x3)
filtered_5x5 = # À compléter
filtered_7x7 = # À compléter
# Afficher les résultats
# À compléter : créer une figure avec 4 images (originale + 3 filtrées)
```

- Méthode 2 : Utilisation de cv2.blur()

```
# Alternative plus simple ´
filtered_3x3 = cv2.blur(img, (3, 3))
filtered_5x5 = cv2.blur(img, (5, 5))
filtered_7x7 = cv2.blur(img, (7, 7))
```

1. Comparez visuellement les résultats avec les trois tailles de filtres.
2. Calculez le PSNR (Peak Signal-to-Noise Ratio) pour chaque filtre si possible.

4. Filtre Gaussian

- Comparer le filtre Gaussien avec le filtre moyenneur : Le filtre Gaussien donne plus de poids aux pixels centraux qu'aux pixels périphériques.

```
import cv2
import matplotlib.pyplot as plt
# Charger l'image bruitée
img = cv2.imread('noisy_image.png', 0)

# Appliquer différents filtres Gaussiens
gaussian_3x3 = cv2.GaussianBlur(img, (3, 3), 0)
gaussian_5x5 = cv2.GaussianBlur(img, (5, 5), 0)
gaussian_7x7 = cv2.GaussianBlur(img, (7, 7), 0)

# Appliquer les filtres moyenneurs correspondants pour comparaison
average_3x3 = cv2.blur(img, (3, 3))
average_5x5 = cv2.blur(img, (5, 5))
average_7x7 = cv2.blur(img, (7, 7))

# À compléter : afficher une comparaison côte à côte # Ligne 1 : filtres moyenneurs
# Ligne 2 : filtres Gaussiens
```

1. Tester différents sigma : 1, 5, et 10.

5. Filtre Médian

Tester l'efficacité du filtre médian contre le bruit poivre et sel.

```
import cv2
import matplotlib.pyplot as plt
# Charger l'image bruitée
img = cv2.imread('noisy_image.png', 0)

# Appliquer le filtre médian avec différentes tailles median_3 = cv2.medianBlur(img, 3)
median_5 = # A compléter median_7 = # A compléter
# Comparer avec filtre moyenneur et Gaussien
average_5 = # A compléter
gaussian_5 = # A compléter
# À compléter : créer une figure comparative
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
# Afficher : original, médian 3, médian 5, médian 7, moyenneur 5, gaussien 5
```

2. Testez le filtre médian sur une image avec du bruit gaussien. Est-il aussi efficace ?